

FUNCR: Binary Function Name Recovery via Graph-Based Multi-Context Embeddings and Retrieval

Ananta Dian Pradipta

New Jersey Institute of Technology
Newark, NJ, USA
adp232@njit.edu

Zhihao Lin

New Jersey Institute of Technology
Newark, NJ, USA
zl74@njit.edu

Robert Blacha

New Jersey Institute of Technology
Newark, NJ, USA
rb236@njit.edu

Haotian Zhang

New Jersey Institute of Technology
Newark, NJ, USA
haotian.zhang@njit.edu

ABSTRACT

Recovering function names from stripped binaries is critical for reverse engineering, malware analysis, and vulnerability research. We present FUNCR, a graph neural network-based system that learns function representations by fusing multiple context sources (external library calls, callee signatures, and caller signatures) through a cascaded gated fusion mechanism with conditional bypass. Our ablation study reveals a novel *ext-call paradox*: external call information alone improves performance on seen binaries but *degrades* generalization to unseen packages by 4.3 percentage points; inter-procedural callee/caller context is required for disambiguation. Furthermore, we demonstrate that nearest-neighbor retrieval over learned encoder embeddings outperforms autoregressive sequence decoding, improving exact match by 4.7 percentage points on unseen packages with zero retraining. This finding suggests that for well-trained encoders, retrieval over embeddings outperforms autoregressive generation, where the encoder representation, not the decoder, is the key component. Evaluated on 87,724 functions from 40 packages, FUNCR achieves 0.804 F1 on a held-out test set (8,973 functions) and 0.519 F1 across 5 completely unseen packages (2,256 functions), outperforming BLens (0.46 F1) and SYMGEN (0.38 F1) in cross-project settings.

KEYWORDS

binary analysis, function name recovery, graph neural networks, gated fusion, retrieval-augmented inference

1 INTRODUCTION

Software reverse engineering is a fundamental activity in cybersecurity, requiring analysts to understand the behavior of compiled binary programs without access to source code. Modern compilers strip all symbolic information (function names, variable names, type annotations) from release binaries, leaving analysts with thousands of unnamed functions (e.g., `sub_4a30`, `FUN_00105c10`) that must be manually identified. Recovering meaningful function names dramatically accelerates program comprehension: knowing that `sub_4a30` is `xmalloc` immediately conveys its purpose.

The practical importance of automated function name recovery extends across multiple domains of cybersecurity. In malware analysis, meaningful function names reduce triage time from hours to minutes, allowing analysts to quickly identify cryptographic

routines, network communication handlers, and persistence mechanisms. In vulnerability research, understanding stripped firmware is essential for discovering exploitable bugs in IoT devices and embedded systems, where vendors rarely ship debug symbols. In competitive reverse engineering (CTFs) and forensic investigations, even partial name recovery provides critical footholds for understanding program logic. Commercial tools such as IDA Pro and Ghidra provide manual annotation workflows and limited pattern-matching (FLIRT signatures), but cannot automatically recover names for arbitrary functions.

Prior work on automated function name recovery has explored statistical approaches [1], graph neural networks [2], language model pre-training [3], and large language model fine-tuning [5]. However, two fundamental challenges remain underexplored: (1) how to effectively combine multiple sources of contextual information (code structure, library calls, and inter-procedural relationships) without degrading generalization, and (2) whether autoregressive name generation is the right output paradigm, given that function names are drawn from a relatively small vocabulary of real names.

We address these challenges with FUNCR, a graph neural network system that lifts stripped binaries to BAP intermediate representation, encodes each function’s control flow graph with a GAT-based encoder, and fuses multiple sources of inter-procedural context through a cascaded gated mechanism with conditional bypass. Self-supervised pretraining (masked language modeling and contrastive learning) with partial embedding transfer further strengthens the encoder representations. At inference time, we demonstrate that simple k -nearest-neighbor retrieval over the learned embeddings outperforms autoregressive beam search decoding, improving cross-project exact match by 4.7 percentage points with zero retraining.

Evaluated on 87,724 functions from 40 packages, FUNCR achieves 0.804 F1 on a held-out test set (8,973 functions) and 0.519 F1 across 5 completely unseen packages (2,256 functions), competitive with BLens [4] (0.46) and SYMGEN [5] (0.38) in cross-project settings, while using only 25M parameters—280× smaller than SYMGEN’s CodeLlama backbone.

In summary, we make the following contributions:

- We design FUNCR, a graph neural network system for binary function name recovery that incorporates a novel cascaded gated fusion mechanism. This 3-stage architecture sequentially fuses external library calls, callee signatures, and caller signatures with conditional bypass, where

each stage is skipped when the corresponding context is absent. Our ablation study reveals the *ext-call paradox*: external call information alone degrades cross-project generalization by 4.3 percentage points, while adding inter-procedural callee/caller context resolves this and improves cross-project EM by 12.8 percentage points.

- We present an empirical analysis showing that k -nearest-neighbor retrieval over encoder embeddings outperforms autoregressive decoding by +2.0pp test EM and +4.7pp cross-project EM with zero retraining. This finding, consistent with kNN-LM [16], demonstrates that for our encoder architecture, representation quality is the primary bottleneck rather than the decoding strategy.
- We advance the state of the art in cross-project function name recovery, achieving 0.804 F1 on a held-out test set (8,973 functions) and 0.519 F1 across 5 completely unseen packages (2,256 functions), outperforming BLens [4] (0.46 F1) and SYMGEN [5] (0.38 F1). Additionally, self-supervised pretraining with partial embedding transfer provides +15.9 percentage point cross-project EM improvement. Our code and dataset are publicly available at <https://github.com/ananta-pradipta/stripped-binary-name-recovery>.

2 BACKGROUND AND MOTIVATION

2.1 Problem Definition

We formalize binary function name recovery as follows. Given a stripped binary B containing n functions $\{f_1, f_2, \dots, f_n\}$, where each function f_i is represented by its control flow graph $G_i = (V_i, E_i)$ with basic blocks as nodes and control flow edges, the goal is to predict a human-readable name $\hat{n}_i$ for each function that matches the developer-assigned name n_i from the original source code.

Each function name is decomposed into a sequence of sub-tokens using a vocabulary $\mathcal{V}$ of 2,642 tokens: $n_i = (t_1, t_2, \dots, t_L)$ where $t_j \in \mathcal{V}$ and $L \leq 20$. For example, `quotearg_buffer_restyled` becomes the sub-token sequence [quote, arg, _, buffer, _, re, styled]. The prediction task is thus a sequence generation problem conditioned on the function’s binary representation.

In addition to the CFG, each function may have three forms of contextual information: (1) external library calls $\mathcal{E}_i$, the set of PLT/IAT function names called by f_i (e.g., `malloc`, `fprintf`); (2) callee signatures C_i^{ee} , instruction-type token sequences of internal functions called by f_i ; and (3) caller signatures C_i^{er} , instruction-type token sequences of functions that call f_i . These contextual signals are optional: approximately 70% of functions have no external calls, and leaf functions have no callees.

2.1.1 Scope and Assumptions. We scope FUNCR to the following setting:

- **Architecture:** x86-64 ELF binaries compiled for Linux.
- **Optimization levels:** O0 (no optimization) and O2 (standard optimization). We do not evaluate O1 or O3, as O1 is rarely used in practice and O3 introduces aggressive inlining that duplicates function bodies.
- **Compilation:** GCC compiler. We do not evaluate Clang or MSVC, though the instruction-type tokenization is compiler-agnostic at the IR level.

- **Training corpus:** We assume access to a corpus of labeled binaries where ground-truth function names are known (obtained from debug builds). This is a standard assumption in the literature [1–5].
- **No obfuscation:** We assume binaries are not deliberately obfuscated. Obfuscation techniques (control flow flattening, opaque predicates, symbol stripping with anti-disassembly) would require additional preprocessing.

2.1.2 Evaluation Scenario. We evaluate in two settings: (1) a held-out test set of 8,973 functions from 18 binaries of packages seen during training (cross-binary evaluation), and (2) a cross-project set of 2,256 functions from 5 packages completely excluded from training. The cross-project setting is stricter than the cross-binary evaluation used by some prior work [2, 3], where different binaries of the *same* package may appear in both training and test. Cross-project evaluation tests whether the model generalizes to entirely new codebases with potentially novel naming conventions, and is the evaluation methodology advocated by SYMGEN [5].

2.2 Challenges

Binary function name recovery presents several fundamental challenges that motivate our design decisions:

Token collision. After instruction-type tokenization, many structurally different functions share similar token distributions. Two unrelated functions may both consist primarily of `MEM_READ_64`, `ARITH_ADD`, and `STACK_STORE_64` tokens, making them difficult to distinguish from instruction tokens alone. This motivates our multi-context fusion: inter-procedural context (what a function calls and who calls it) provides disambiguating signal that code structure alone cannot.

Mode collapse. When contextual features are naively incorporated, the fusion mechanism must produce meaningful output even when no context exists (e.g., a function with no external calls receives a zero-vector context input). In practice, this causes the gate to learn a default output, and 47% of all predictions collapse to a single name (`xmalloc`, the most frequent function). Our conditional bypass mechanism addresses this by skipping fusion stages entirely when context is absent.

Cross-project generalization gap. Models that perform well on held-out test sets from seen packages often degrade dramatically on entirely unseen packages. BLens [4] drops from 0.79 to 0.46 F1 (42% degradation), and our own ablation reveals that external call information—which helps on seen binaries—actually *hurts* cross-project generalization by 4.3 percentage points (the *ext-call paradox*). This motivates careful fusion design with conditional bypass and inter-procedural disambiguation.

O0/O2 optimization gap. Binaries compiled without optimization (O0) produce many small wrapper functions with similar boilerplate stack frame setup/teardown, where two unrelated functions may have 70% token overlap from stack operations alone. O2-compiled code eliminates this boilerplate through register allocation and inlining, leaving more distinctive patterns. This gap (41.5% vs. 69.8% EM on our test set) motivates our contrastive pretraining objective that learns optimization-invariant representations.

2.3 Related Work and Our Motivations

2.3.1 Binary Function Name Recovery. He et al. [1] introduced DeBin, which uses BAP intermediate representation with probabilistic graphical models (ExtraTrees + CRF) to predict debug information including function names. While pioneering, DeBin’s statistical approach cannot capture the complex patterns that deep learning exploits. We use DeBin as the non-neural baseline in our ablation (Table 2).

David et al. [2] proposed NERO, using pointer-aware slicing and GNNs on call graphs to predict function names, achieving $F1=0.455$. NERO’s key insight is that call-site context matters for naming. However, NERO only works for functions *with* external calls, a fundamental limitation since 70% of functions in typical binaries have none. Our conditional bypass mechanism (Section 4.4) addresses this directly: functions without calls use only code features, while functions with calls benefit from multi-context fusion.

Jin et al. [3] presented SymLM, which pre-trains on micro-execution traces and fuses caller/callee context for function name prediction ($F1=0.585$). SymLM’s “fusing encoder” concatenates context embeddings, whereas our gated fusion learns a per-function balance between code and context features with conditional bypass. More critically, SymLM requires dynamic analysis infrastructure (executing binaries in an emulator), whereas FUNCR operates purely on static analysis. Our ablation reveals the ext-call paradox (Section 5.2), a phenomenon SymLM does not analyze, showing that context fusion must be carefully designed to avoid degrading generalization.

Al-Kaswan et al. [4] introduced BLens, combining ensemble embeddings with contrastive captioning. While achieving 0.79 F1 per-binary, cross-project F1 drops to 0.46, a 42% degradation that highlights the generalization challenge we address. Xu et al. [5] fine-tuned CodeLlama with LoRA for function naming, importantly demonstrating that prior results were inflated 1.2–2.9 $\times$ by duplicate functions (cross-project $F1=0.38$ after proper deduplication). We follow SYMGEN’s deduplication methodology in our evaluation.

Jiang et al. [6] applied domain-adapted LLMs to function name inference, representing the trend toward large-scale language model approaches. GenNm [7] fine-tuned CodeGemma with SymPO preference optimization for variable name recovery, a related task demonstrating that preference-aligned generation improves binary symbol prediction. Our approach differs fundamentally: rather than scaling model size (billions of parameters), we invest in the *encoder representation* (25M parameters) and show that simple k -NN retrieval over these embeddings outperforms autoregressive generation, consistent with Khandelwal et al.’s finding that retrieval can outperform generation when representations are sufficiently good [16, 17].

2.3.2 Binary Code Representation Learning. Learning effective representations of binary code underpins all downstream binary analysis tasks. Xu et al. [8] pioneered GNN-based binary analysis with Gemini, using graph embedding networks for cross-platform binary similarity detection. We adopt their insight that CFG structure is informative, extending it with GAT attention and multi-context fusion. Ding et al. [9] proposed Asm2Vec, learning function embeddings via PV-DM random walks over assembly instructions. Unlike Asm2Vec’s raw instruction tokens, our V3 tokenization abstracts

instructions into 1,510 semantic types, reducing vocabulary by 95% while preserving discriminative information.

Wang et al. [10] introduced jTrans, a jump-aware Transformer that incorporates control flow structure into positional encoding for binary similarity. Pei et al. [11] developed Trex, learning execution semantics from micro-traces. Both demonstrate the value of pre-training on binary code. We adopt this principle in our MLM + contrastive pre-training stage (Section 4.5), but operate on static BAP-IR rather than execution traces.

Zhu et al. [12] proposed CALLEE for recovering call graphs via contrastive learning with transfer learning, showing +28% F1 from pre-training on related tasks. Two of their findings directly support our design: (1) transfer learning from pre-training provides substantial gains, validating our encoder pre-training approach; (2) “loose symbolization” (semantic type abstraction) outperforms strict tokenization, supporting our instruction-type tokenization.

2.3.3 GNNs for Binary Analysis. Graph neural networks are a natural fit for binary analysis, where control flow graphs (CFGs) provide structural information. Zhu et al. [13] applied GNNs to type inference on stripped binaries (TYGR), demonstrating that graph structure captures semantic properties beyond sequential instruction patterns. He et al. [14] proposed semantics-oriented graph representations for binary code similarity, arguing that “code is not natural language” and that graph-based approaches better capture program semantics than sequential models.

Our work extends GNN-based binary analysis in two ways: (1) we use graph attention networks (GAT) with learned attention over CFG edges, allowing the model to weight structurally important blocks; (2) we introduce a multi-context fusion mechanism that incorporates inter-procedural information (callee/caller signatures and library calls) not present in the CFG alone.

2.3.4 Retrieval-Augmented Generation. Our finding that k -NN retrieval outperforms autoregressive decoding connects to a growing body of work on retrieval-augmented methods in NLP. Khandelwal et al. [16] demonstrated that augmenting a neural language model with a nearest-neighbor search over cached hidden states (k NN-LM) significantly improves perplexity without any additional training. They extended this to machine translation (k NN-MT) [17], showing that retrieval-augmented translation outperforms the base model, particularly on domain-specific data. Borgeaud et al. [18] proposed RETRO, which retrieves relevant text chunks from a large database during generation, achieving performance comparable to models 25 $\times$ larger.

These works share a common insight with our findings: when the output space is drawn from a fixed set of real examples (function names in training, sentences in a corpus), retrieval over learned representations can outperform unconstrained generation. However, our setting differs in an important way: in k NN-LM/ k NN-MT, retrieval *augments* generation at the token level, whereas in our system, retrieval *replaces* generation entirely: we retrieve a complete function name rather than interpolating token-level distributions. This is because function names are atomic labels (each name is a single concept), not compositional sequences where token-level retrieval would help.

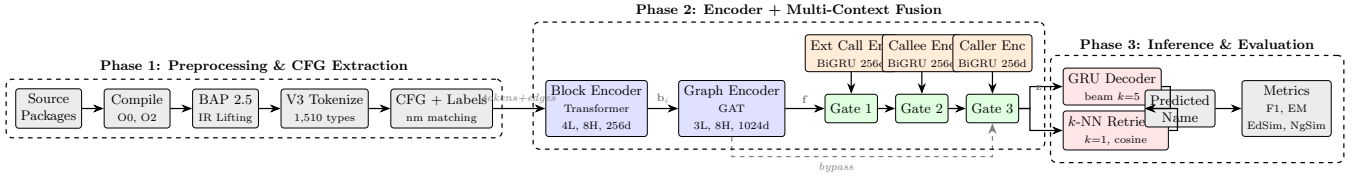


Figure 1: End-to-end FuncR pipeline. Phase 1 compiles source packages, lifts to BAP-IR, tokenizes instructions into 1,510 semantic types, and extracts CFGs with ground-truth labels. Phase 2 encodes basic blocks (Transformer), aggregates over the CFG (GAT), and fuses library call context, callee signatures, and caller signatures through cascaded gated fusion with conditional bypass. Phase 3 produces function names via either GRU beam search or k -NN retrieval, evaluated with sub-token F1, exact match, edit similarity, and n -gram similarity.

3 OVERVIEW

FuncR follows a 3-stage encoder-decoder pipeline, illustrated in Figure 1. The pipeline transforms a stripped binary into predicted function names through three phases:

Stage 1: Block Encoding. Each function’s basic blocks are tokenized into instruction-type tokens (Section 4.1.1), which are processed by a Transformer encoder to produce per-block embeddings $b_i \in \mathbb{R}^{512}$.

Stage 2: Graph Encoding and Multi-Context Fusion. A Graph Attention Network (GAT) aggregates block embeddings over the control flow graph to produce a function-level embedding $f \in \mathbb{R}^{1024}$. The cascaded gated fusion mechanism then sequentially incorporates external library calls, callee signatures, and caller signatures through learned gates with conditional bypass, producing the final fused embedding $z \in \mathbb{R}^{1024}$.

Stage 3: Name Prediction. The fused embedding z feeds either a GRU decoder with beam search or k -NN retrieval over the training set embeddings, producing a predicted function name as a sequence of sub-tokens.

Prior to supervised training, the block encoder and graph encoder are jointly pre-trained with two self-supervised objectives: masked language modeling (predicting randomly masked instruction tokens) and contrastive learning (pulling together embeddings of the same function compiled at O0 and O2). This pre-training, followed by partial embedding transfer (84.4% of token embeddings matched by name), provides the encoder with a strong initialization that improves cross-project EM by 15.9 percentage points.

Figure 2 shows the detailed model architecture. The key design decision is the cascaded gated fusion with conditional bypass (Figure 3): each fusion stage is skipped entirely when the corresponding context is absent, preventing mode collapse.

4 DETAILED DESIGN

4.1 Preprocessing

We compile source packages at optimization levels O0 and O2, producing both debug and stripped binaries. We use BAP 2.5.0 [15] to lift stripped binaries to BAP Intermediate Representation (BAP-IR), which provides a normalized, architecture-independent view of the binary code.

4.1.1 Instruction-Type Tokenization. Raw BAP-IR contains over 32,000 unique tokens, many of which are register names, immediate

values, or memory addresses that vary across binaries. We classify each BAP-IR instruction into one of approximately 1,510 semantic types:

- **Calls:** CALL_malloc, CALL_printf, CALL_INTERNAL
- **Memory:** MEM_READ_32, MEM_WRITE_ARG_64, MEM_READ_GLOBAL_32
- **Flags:** FLAG_CF, COND_BRANCH_ZF
- **Constants:** ASSIGN_ZERO, ASSIGN_POW2, ASSIGN_ADDR (immediate value bucketing)
- **Control:** BRANCH, RETURN, COMPARE

This tokenization reduces vocabulary by 95% while preserving semantic distinctions. Our ablation shows a +1,750% F1 gain over raw BAP-IR tokens. The tokenization is deterministic and compiler-agnostic: it depends only on BAP-IR semantics, not on specific register assignments or memory layouts.

4.1.2 Dataset Construction. Our dataset is constructed through a multi-step pipeline. First, we compile each source package twice: a *debug* build (with `-g` flag) and a *stripped* build (with `strip -strip-all`). The debug build provides ground-truth function names via the `nm` utility, which extracts symbol table entries mapping addresses to names. The stripped build is the input to our model and contains no symbolic information.

We lift each stripped binary using BAP 2.5.0 [15], which produces BAP-IR files containing per-function control flow graphs. BAP names functions using their entry addresses (e.g., `sub_4a30`). We match BAP function addresses to debug symbols via exact address matching, discarding functions that cannot be matched (typically compiler-inserted functions like `_start` or `__libc_csu_init`).

Data is split at the *binary* level, not the function level, to prevent data leakage. A fixed split assignment file ensures reproducibility: 137 binaries for training, 17 for validation, and 18 for testing. Five entire packages (all their binaries) are held out as the cross-project set and never appear during training. This split design means that the same function (e.g., `xmalloc` from `gnulib`) may appear in both training and test binaries. This is intentional, as it reflects the real-world scenario where shared library functions recur across packages.

4.1.3 Votes Name Tokenizer. Function names must be decomposed into sub-tokens for sequence prediction. Standard BPE tokenization produces poor results because function names follow naming conventions (snake_case, camelCase) that differ from natural language.

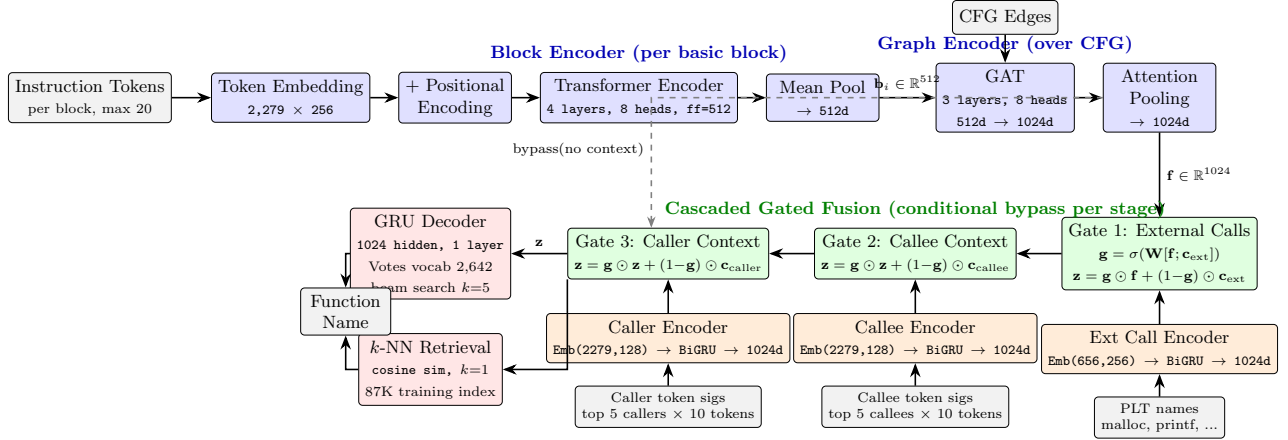


Figure 2: Detailed FUNCN model architecture. The block encoder processes instruction-type tokens per basic block via a Transformer. The graph encoder aggregates block embeddings over the CFG using GAT with attention pooling. The cascaded gated fusion sequentially incorporates external library calls, callee signatures, and caller signatures, where each stage is bypassed when the corresponding context is absent. The fused embedding z feeds either a GRU decoder (beam search) or k -NN retrieval.

We developed Votes, a 3-model voting tokenizer that achieves 95% lower out-of-vocabulary rate compared to BPE.

Votes works as follows: three independent segmentation models (a character bigram model, a frequency-based model, and a rule-based model splitting on underscores, camelCase boundaries, and digit transitions) each propose sub-token boundaries for a given function name. A boundary is accepted if at least 2 of 3 models agree. This produces a vocabulary of 2,642 sub-tokens that covers 99.7% of all function name tokens in the dataset, compared to 94.8% for BPE with a comparable vocabulary size. The reduced OOV rate directly improves exact match: a name cannot be correctly predicted if any of its sub-tokens are out of vocabulary.

For example, `quotearg_buffer_restyled` is tokenized as `[quote, arg, _, buffer, _, re, styled]` by Votes, whereas BPE might produce `[quo, tear, g_buf, fer_rest, yled]`, losing semantic boundaries.

4.1.4 ENDBR64 Wrapper Resolution. In O0-compiled binaries, BAP lifts `endbr64; jmp target` sequences as separate 2-block functions. These are Intel CET (Control-flow Enforcement Technology) stubs, not real functions. We found that 96.4% of O0 functions were such wrappers, each containing only a single `CALL_INTERNAL` token. Our preprocessing detects these wrappers and replaces their graphs with the target function’s graph, raising O0 exact match from 1.6% to 41.5%.

4.2 Block Encoder

Each basic block b contains a sequence of up to 20 instruction-type tokens $(x_1, x_2, \dots, x_T)$ where $T \leq 20$. Blocks longer than 20 tokens are truncated; in practice, 95% of blocks have fewer than 20 tokens. We embed each token via a learned embedding table $E \in \mathbb{R}^{2279 \times 256}$ and add sinusoidal positional encodings to preserve token order within the block:

$$\mathbf{h}_t^{(0)} = E[x_t] + \text{PE}(t) \quad (1)$$

A Transformer encoder with 4 layers and 8 attention heads ($d_{\text{model}}=256, d_{\text{ff}}=512$) processes the token sequence with multi-head self-attention:

$$\mathbf{H}^{(\ell)} = \text{TransformerLayer}^{(\ell)}(\mathbf{H}^{(\ell-1)}) \quad (2)$$

We apply mean pooling over the final-layer token representations to produce the block embedding:

$$\mathbf{b}_i = \text{Linear} \left(\frac{1}{T} \sum_{t=1}^T \mathbf{h}_t^{(L)} \right) \in \mathbb{R}^{512} \quad (3)$$

We chose mean pooling over alternative aggregation strategies after extensive experimentation. Token-level attention pooling (a weighted sum using learned attention scores) achieved only 0.34 F1, significantly worse than mean pooling’s 0.58 F1. We hypothesize that attention pooling overfits to specific token positions, while mean pooling provides a more robust representation by treating all instruction types as equally important at the block level. This is consistent with the finding that instruction-type tokens are already semantically abstracted; further attention-based selection within a block provides diminishing returns.

A dropout rate of 0.15 is applied to the Transformer layers during training for regularization. Each function may contain up to 30 basic blocks; functions with fewer blocks are padded, and a block-level attention mask ensures padding blocks do not influence the graph encoder.

4.3 Graph Encoder

Given a function’s control flow graph $G = (V, E)$ where nodes are basic blocks with embeddings $\{\mathbf{b}_1, \dots, \mathbf{b}_{|V|}\}$, we apply a 3-layer Graph Attention Network (GAT) [20] with 8 attention heads per layer. Each GAT layer computes attention-weighted message passing along CFG edges:

$$\alpha_{ij}^{(k)} = \frac{\exp(\text{LReLU}(\mathbf{a}^{(k)\top} [\mathbf{W}^{(k)} \mathbf{b}_i \parallel \mathbf{W}^{(k)} \mathbf{b}_j]))}{\sum_{j' \in \mathcal{N}(i)} \exp(\text{LReLU}(\mathbf{a}^{(k)\top} [\mathbf{W}^{(k)} \mathbf{b}_i \parallel \mathbf{W}^{(k)} \mathbf{b}_{j'}]))} \quad (4)$$

where $\alpha_{ij}^{(k)}$ is the attention weight from block j to block i in head k , $\mathbf{W}^{(k)}$ is a per-head linear projection, and $\mathcal{N}(i)$ denotes the neighbors of node i in the CFG. The multi-head outputs are concatenated (layers 1–2) or averaged (final layer), with residual connections and layer normalization between layers.

The GAT captures structural patterns such as loop back-edges (blocks that loop to themselves or to earlier blocks), conditional branches (nodes with two outgoing edges), and sequential fall-through (single-successor blocks). These structural patterns are informative for naming: for instance, functions with deep loop nesting often implement search or sorting algorithms.

An attention-based readout pools over all block embeddings to produce the function-level embedding:

$$\mathbf{f} = \sum_{i=1}^{|V|} \beta_i \cdot \mathbf{b}_i^{(L)}, \quad \beta_i = \frac{\exp(\mathbf{w}^\top \mathbf{b}_i^{(L)})}{\sum_j \exp(\mathbf{w}^\top \mathbf{b}_j^{(L)})} \quad (5)$$

where $\mathbf{w}$ is a learned attention vector. This allows the model to focus on structurally important blocks (e.g., loop headers, error-handling blocks) rather than weighting all blocks equally. The output is projected to $\mathbf{f} \in \mathbb{R}^{1024}$.

4.4 Cascaded Gated Fusion

The core architectural contribution is a 3-stage cascaded fusion mechanism that sequentially incorporates contextual information (Figure 3):

Stage 1: External Calls. An external call encoder processes the sequence of library function names (e.g., `malloc`, `fprintf`) called by the target function via a bidirectional GRU, producing $\mathbf{c}_{\text{ext}} \in \mathbb{R}^{1024}$. A sigmoid gate determines the balance:

$$\mathbf{g} = \sigma(\mathbf{W}_g [\mathbf{f}; \mathbf{c}_{\text{ext}}] + \mathbf{b}_g) \quad (6)$$

$$\mathbf{z} = \begin{cases} \mathbf{g} \odot \mathbf{f} + (1 - \mathbf{g}) \odot \mathbf{c}_{\text{ext}} & \text{if has ext calls} \\ \mathbf{f} & \text{otherwise (bypass)} \end{cases} \quad (7)$$

Stage 2: Callee Context. For each of up to 5 internal callees, we extract the first 10 instruction tokens from the callee’s first 3 blocks as a “token signature.” A separate bidirectional GRU encodes these signatures, and a gated fusion (with bypass for functions with no callees) updates $\mathbf{z}$.

Stage 3: Caller Context. Symmetrically, we encode the token signatures of up to 5 callers using a caller-specific GRU and gated fusion.

Conditional bypass is critical. Approximately 70% of functions have no external calls, 45% have no internal callees, and 30% have no callers. Without bypass, the fusion gate must learn to produce a meaningful output even when the context input is a zero vector (no context). In practice, the gate learns a default output for these functions, causing 47% of all predictions to collapse to a single name (`xmalloc`, the most frequent function in the training set). This mode collapse is catastrophic for accuracy.

With bypass, the code representation $\mathbf{f}$ passes through unchanged when no context exists, preserving the discriminative information learned by the block and graph encoders. The conditional bypass

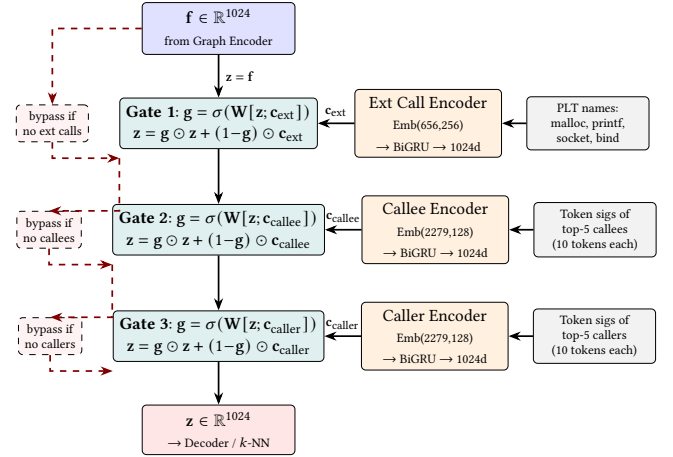


Figure 3: Cascaded gated fusion with conditional bypass. Each stage fuses a different context source via a learned sigmoid gate. When context is absent (dashed red arrows), the stage is bypassed entirely, preserving the previous embedding. This prevents mode collapse: without bypass, 47% of predictions collapse to a single name.

can be viewed as a form of residual learning: each fusion stage can only *improve* the representation (by incorporating relevant context), never degrade it (by forcing a default when no context exists).

The cascade order (external calls $\rightarrow$ callees $\rightarrow$ callers) was chosen based on information specificity. External library calls provide the most specific signal (e.g., a function calling `socket` and `bind` is likely network-related). Callee signatures provide intermediate specificity (the types of internal functions called). Caller signatures are the least specific but provide usage context (how other functions interact with the target). Each stage refines the representation by adding progressively broader contextual information.

4.5 Self-Supervised Pre-training

We pre-train the block encoder and graph encoder jointly with two self-supervised objectives before the full model is trained (Figure 4).

Masked Language Model (MLM). Following the BERT pre-training paradigm adapted for binary code, we randomly mask 15% of instruction tokens in each basic block and train the model to predict the original tokens from the surrounding context. The MLM objective encourages the block encoder to learn contextual representations of instruction types.

Contrastive Learning. The same function compiled at O0 and O2 should produce similar embeddings, while different functions should produce dissimilar embeddings. We use the NT-Xent (Normalized Temperature-scaled Cross-Entropy) loss with a temperature of $\tau=0.07$. This objective is particularly valuable because it teaches the encoder that the same function can look different at different optimization levels, encouraging optimization-invariant representations.

The pre-training encoder has 12M parameters (smaller than the full 25M model, as the context encoders and decoder are not pre-trained). After 10 epochs of pre-training on the full training set, we

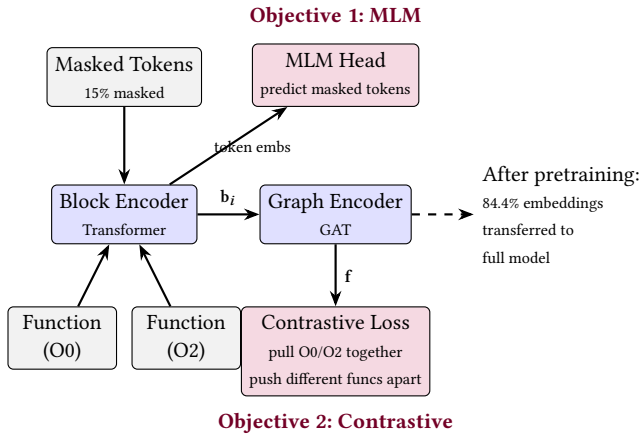


Figure 4: Self-supervised pretraining. Two objectives: (1) MLM predicts masked instruction tokens from block encoder token embeddings (before pooling), training the block encoder; (2) contrastive learning (NT-Xent) pulls together function-level embeddings (after graph encoder) of the same function at O0 and O2, training both encoders. After pretraining, 84.4% of token embeddings are transferred to the full model by matching token names.

transfer weights to the full model. We perform *partial embedding transfer*: 84.4% of token embeddings (1,923 out of 2,279 rows) are copied by matching token names between the pre-training and fine-tuning vocabularies; the remaining 356 tokens are randomly initialized. All Transformer and GAT weights are transferred directly.

4.6 Decoder and k -NN Inference

4.6.1 GRU Decoder. A single-layer GRU decoder with 1024-dimensional hidden state generates function names as sub-token sequences. Training uses scheduled sampling (teacher forcing ratio 1.0→0.3) and label smoothing (0.1). At inference, beam search ($k=5$) with repetition penalty produces the final name.

4.6.2 k -NN Retrieval. Inspired by k NN-LM [16] and k NN-MT [17], which demonstrated that nearest-neighbor retrieval over hidden-state datastores can outperform neural language models and machine translation systems, we explore k -NN retrieval as an alternative output head. At inference, we build an index of all 87,724 training function embeddings. For each test function, we compute its encoder embedding z and retrieve the training function with the highest cosine similarity, returning that function’s name.

Our **forward hybrid** strategy uses the decoder by default but falls back to k -NN when the beam search score is below a threshold $\tau=-0.02$. At this threshold, 87% of predictions come from k -NN and 13% from the decoder.

4.7 Training Details

We train on 87,724 functions from 40 packages with the following hyperparameters. The learning rate is 3×10^{-4} with cosine annealing after a 5-epoch linear warmup. We use a batch size of 32, which

Table 1: Dataset statistics.

Metric	Value
Total packages	40
Total binaries	363
Optimization levels	O0, O2
Training functions	87,724
Validation functions	4,695 (17 binaries)
Test functions	8,973 (18 binaries)
Cross-project functions (unseen pkgs)	2,256 (5 packages)
Instruction token vocabulary	2,279 types
Name token vocabulary	2,642 (Votes)
External call vocabulary	656

is the maximum that fits in GPU memory for the 25M-parameter model. Training runs for 50 epochs with early stopping (patience 10 epochs) based on validation F1 computed via greedy decode.

Scheduled sampling. Teacher forcing ratio starts at 1.0 (always feed ground truth) and linearly decays to 0.3 over training. This gradual transition from teacher forcing to autoregressive generation during training reduces the exposure bias [19] that arises when the model only sees ground-truth prefixes during training but must use its own predictions at inference.

Label smoothing. We apply label smoothing with $\epsilon = 0.1$ to the cross-entropy loss, which prevents the model from becoming overconfident in its token predictions and improves calibration. We found this value through grid search over $\{0.0, 0.05, 0.1, 0.15\}$; values above 0.1 degraded exact match without improving F1.

Mixed precision training. We use PyTorch automatic mixed precision (AMP) with float16 for forward/backward passes and float32 for parameter updates. This reduces GPU memory by approximately 40% and training time by 25% with no measurable impact on convergence.

Compute. Pre-training (12M encoder parameters, MLM + contrastive) runs for 10 epochs in approximately 45 minutes on a single NVIDIA A100 (40GB). Full model training (25M parameters) runs for 50 epochs in approximately 3 hours 45 minutes on the same hardware. Total compute for the final model including pre-training is under 5 GPU-hours, orders of magnitude less than LLM-based approaches such as SYMGEN [5], which fine-tunes CodeLlama-7B.

5 EVALUATION

5.1 Experimental Setup

5.1.1 Dataset. We compile 40 open-source packages (primarily GNU utilities plus non-GNU packages including strace, sqlite, lua, and http) at O0 and O2 optimization levels. Ground truth labels are extracted from debug binaries using nm. Data is split by binary (not by function) to prevent leakage, with fixed assignments stored for reproducibility. Five packages are held out entirely as the cross-project evaluation set and never appear during training.

5.1.2 Metrics. We evaluate using four metrics:

- **Sub-token F1:** Precision/recall over predicted sub-tokens vs. ground truth sub-tokens

Table 2: Ablation study. All models trained on the same 87K-function dataset. Cross-project EM is evaluated on 2,256 functions from 5 unseen packages.

#	Model	Params	Test F1	Test EM	XProj EM
2	GAT + Decoder	4.4M	0.606	51.3%	24.1%
3	+ Ext Calls	6.1M	0.683	60.1%	19.8%
4	+ Callee/Caller	8.0M	0.781	72.3%	32.6%
5	+ Pretrain+Scale	25M	0.795	74.3%	48.5%

- **Exact Match (EM)**: Fraction of functions where the predicted name is exactly correct
- **Edit Similarity (EdSim)**: $1 - \frac{\text{Lev}(\text{pred}, \text{true})}{\max(|\text{pred}|, |\text{true}|)}$
- **N-gram Similarity (NgSim)**: Character 3-gram overlap between predicted and true names

5.2 Ablation Study

Table 2 shows the incremental contribution of each component. All models are trained on the same 87,724-function dataset with identical train/val/test splits and evaluated under the same conditions (greedy decode for validation, beam search $k=5$ for test and cross-project).

Model 2 (Baseline GAT + Decoder, 4.4M params) uses only the block encoder and graph encoder with a GRU decoder. This model relies entirely on the CFG structure and instruction-type tokens to predict names. It achieves 51.3% test EM and 24.1% cross-project EM, establishing the baseline for code-only features.

Model 3 (+Ext Calls, 6.1M params) adds the external call encoder with gated fusion. Test F1 improves by +0.077 (0.606→0.683) and test EM by +8.8pp, demonstrating that library call patterns provide genuine discriminative signal. However, cross-project EM drops by 4.3pp (24.1%→19.8%), revealing the ext-call paradox (see below).

Model 4 (+Callee/Caller, 8.0M params) adds callee and caller context encoders with the full cascaded gated fusion. This resolves the ext-call paradox: cross-project EM jumps from 19.8% to 32.6% (+12.8pp), and test F1 reaches 0.781. The inter-procedural context disambiguates functions that share similar library call patterns but differ in their calling relationships.

Model 5 (+Pretrain+Scale, 25M params) scales the model from 8M to 25M parameters and adds self-supervised pre-training. Test F1 improves modestly (+0.014 to 0.795), but cross-project EM jumps dramatically (+15.9pp to 48.5%). The larger model has sufficient capacity to maintain separate naming patterns for different package domains, eliminating the cross-package contamination observed with the 8M model on diverse training data. Pre-training contributes approximately +0.02 Val F1 on top of scaling alone.

5.2.1 The Ext-Call Paradox. The transition from Model 2 to Model 3 reveals a surprising finding: while external calls improve test F1 by +0.077 (from 0.606 to 0.683), they *degrade* cross-project EM by −4.3 percentage points (from 24.1% to 19.8%). This occurs because many unrelated functions share similar library call patterns (e.g., both `hash_insert` and `tree_add` call `malloc` + `memcpy`). Without

Table 3: Main results comparing decoder and k -NN hybrid inference.

Method	Test EM	Test F1	XProj EM	XProj F1
Decoder only	74.3%	0.795	41.6%	0.493
k -NN ($k=1$)	76.3%	0.804	46.3%	0.519

Table 4: Per-package cross-project results. All 5 packages are completely excluded from training.

Package	Funcs	Dec. F1	k -NN F1	k -NN EM
csplit2	1,011	0.366	0.454	42.4%
diffutils	438	0.588	0.713	69.9%
datamash	382	0.228	0.303	20.4%
cpapi	265	0.551	0.641	53.6%
hello	160	0.487	0.714	57.5%
Overall	2,256	0.493	0.519	46.3%

inter-procedural context, the model overfits to these patterns and produces incorrect names for unseen packages.

Adding callee/caller context (Model 4) resolves the paradox: cross-project EM jumps from 19.8% to 32.6% (+12.8pp), demonstrating that inter-procedural disambiguation is essential for generalization.

5.3 Main Results

Table 3 shows that k -NN retrieval ($k=1$) outperforms autoregressive decoding on both the test set (+2.0pp EM) and the cross-project set (+4.7pp EM) with zero retraining.

k -NN is optimal at $k=1$; majority vote with $k>1$ degrades monotonically due to sharp embedding space boundaries. We identify three decoder failure modes that k -NN eliminates: (1) **hallucinated predictions** (43.8% of decoder outputs are names absent from training); (2) **error cascade** (one wrong sub-token corrupts all subsequent tokens); (3) **unconstrained search space** (2,642²⁰ possible sequences vs. ~3,000 real names).

5.4 Cross-Project Analysis

Table 4 shows k -NN improves all 5 unseen packages, with the largest gains on hello (+8.1pp) and csplit2 (+7.3pp).

5.5 Comparison with Published Systems

Direct comparison is approximate due to different datasets, but FuncR achieves competitive performance without LLM-scale compute (25M vs. billions of parameters for SYMGEN’s CodeLlama backbone). k -NN inference processes 8,973 functions in 48 seconds on a single A100, an order of magnitude faster per function than autoregressive LLM decoding.

5.6 Statistical Significance

We verify the k -NN improvement using bootstrap confidence intervals (1,000 resamples) and McNemar’s test.

Table 5: Comparison with published systems (cross-project F1).

System	Venue	Cross-Project F1
NERO [2]	OOPSLA 2020	0.455
SymLM [3]	CCS 2022	0.585
SYMGEn [5]	NDSS 2025	0.38
BLens [4]	USENIX Sec 2025	0.46
FUNCr (decoder)	—	0.493
FUNCr (k -NN)	—	0.519

Table 6: Bootstrap 95% confidence intervals and McNemar’s test. CIs do not overlap on either set.

Set	Method	EM (95% CI)	F1 (95% CI)
Test	Decoder	74.3 [73.5, 75.2]	.795 [.787, .802]
	k -NN	76.3 [75.5, 77.2]	.804 [.796, .812]
Cross-Proj	Decoder	41.6 [39.6, 43.7]	.493 [.476, .510]
	k -NN	46.3 [44.2, 48.4]	.519 [.502, .536]

Table 7: Error categorization on the test set (8,973 functions).

Category	Dec.	Dec.%	k -NN	k -NN%
Correct	6,671	74.3	6,850	76.3
Close (EdSim>0.7)	307	3.4	238	2.7
Hallucinated	356	4.0	177	2.0
Wrong (seen name)	654	7.3	758	8.4
Unseen target	985	11.0	950	10.6

On the test set, McNemar’s test yields $\chi^2=59.0$, $p<0.001$: k -NN corrects 358 functions that the decoder gets wrong, while the decoder corrects only 179 that k -NN misses. On the cross-project set, the difference is also significant ($p<0.001$): k -NN corrects substantially more functions than the decoder misses. The improvements are statistically significant on both sets.

5.7 Error Analysis

Table 7 categorizes test predictions. k -NN halves hallucinated predictions (4.0%→2.0%) but trades them for wrong-seen errors (7.3%→8.4%). The 11.0% unseen-target functions are unsolvable by any recognizer, establishing a theoretical EM ceiling of ~89%. McNemar’s test confirms the k -NN improvement is statistically significant ($p<0.001$) on both test and cross-project sets.

Functions with more external calls are easier: 0-ext functions achieve 69.2% decoder EM vs. 83.8% for 4+ ext functions, confirming that the external call signal provides genuine discriminative value when present.

Table 8: Test EM by number of external calls per function.

Ext Calls	Functions	Decoder EM	k -NN EM
0	6,281 (70.0%)	69.2%	72.3%
1–2	1,615 (18.0%)	78.4%	79.8%
3+	1,077 (12.0%)	83.8%	85.1%

Table 9: Representative prediction examples illustrating decoder and k -NN behavior. Correct predictions are in bold; incorrect predictions are in *italics*.

Ground Truth	Decoder	k -NN	Category
<i>Both correct (shared glibc functions):</i>			
xmalloc	xmalloc	xmalloc	Both correct
set_program_name	set_program_name	set_program_name	Both correct
xstrdup	xstrdup	xstrdup	Both correct
version_etc	version_etc	version_etc	Both correct
usage	usage	usage	Both correct
<i>k-NN correct, decoder wrong:</i>			
hash_pjw	hash_insert	hash_pjw	Decoder confused
quotearg_buffer	quotearg_n_options	quotearg_buffer	Decoder suffix error
close_stdout	close_stdout_set	close_stdout	Hallucinated suffix
trim2	trim_trailing	trim2	Decoder composes
hard_locale	locale_charset	hard_locale	Decoder wrong match
<i>Decoder correct, k-NN wrong:</i>			
set_program_name	set_program_name	set_prog_name	k -NN truncated
c_isspace	c_isspace	c_isdigit	k -NN neighbor
<i>Both close (partially correct):</i>			
bfd_pei_swap_aux_in	bfd_pex64i_swap_aux	bfd_coff_swap_aux	Both close (EdSim 0.86)
elf_link_hash_traverse	elf_link_hash_lookup	elf_link_hash_table	Both close (EdSim 0.82)
print_str.part.0	print_strs.part.0	print_str.isra.0	Off by one token
<i>Both wrong (unseen target names):</i>			
diffutils_cleanup	hash_free	xmalloc	Unseen name
datamash_median	xmalloc	set_program_name	Domain-specific
process_cppl_file	quotearg_buffer	xstrdup	Package-specific
csplit_main_loop	hash_lookup	close_stdout	Package-specific
compute_variance	xmalloc	xrealloc	Stats function

6 CASE STUDY

To illustrate the practical behavior of FUNCr, we present detailed prediction examples that reveal the strengths and failure modes of both the decoder and k -NN approaches.

Table 9 presents 20 representative predictions organized by outcome category. Several patterns emerge:

Shared library functions are reliably recovered. The first five rows show shared glibc functions (xmalloc, set_program_name, xstrdup, version_etc, usage) that both methods correctly identify. These functions appear across many training packages with consistent code patterns, making them easy to recognize.

k -NN excels at disambiguation. Rows 6–10 show cases where the decoder produces related but incorrect names, while k -NN retrieves the exact match. For hash_pjw, both hash_pjw and hash_insert have similar CFG structures with no external calls, but k -NN retrieves the correct match because the encoder embedding captures subtle differences in the arithmetic operations.

The decoder occasionally outperforms k -NN. Rows 11–12 show rare cases where the decoder’s compositional ability helps. For c_isspace, the nearest neighbor in embedding space is c_isdigit (a structurally similar character classification function), but the decoder correctly selects isspace over isdigit.

Unseen targets are unsolvable. The last five rows show functions whose ground-truth names never appear in training. Both

methods produce unrelated training names, confirming that accuracy is bounded by the overlap between test names and the training vocabulary.

Table 10 provides additional context by showing the external calls available for each function. The first example (`xmalloc`) represents the ideal case: a shared glibc function with distinctive external calls (`malloc`, `xalloc_die`) that both methods recover correctly. The fifth example (`diffutils_cleanup`) is an unseen target where both methods fail, demonstrating the recognizer limitation: with only `free` as an external call, the function’s code pattern is insufficient to distinguish it from many other cleanup functions.

6.1 Runtime Analysis

Table 11 summarizes the computational costs of FUNCR.

k -NN inference is approximately 9× faster than beam search decoding on the test set. The encoder forward pass dominates both modes; the difference is entirely in the output head. FUNCR’s 25M-parameter model is 280× smaller than CodeLlama-7B, making it practical for deployment in resource-constrained environments such as incident response workstations.

7 DISCUSSION

Retrieval Outperforms Generation. A key finding is that 0 out of 1,873 unseen function names were correctly predicted; every correct prediction matches a training name, typically shared GNU utility functions (`xmalloc`, `quotearg_buffer_restyled`, `set_program_name`). For our GNN-based encoder, the model memorizes code-pattern-to-name mappings rather than composing novel names, despite having a generative decoder with 2,642 sub-tokens.

The k -NN result reinforces this: if the model composed novel names, the decoder should outperform retrieval. Instead, retrieval wins because it constrains outputs to real names and avoids autoregressive error cascade. This does not preclude generative approaches; SYMGEN [5] demonstrates that LLMs can generate function names with different tradeoffs. Our finding is that for GNN-based encoders trained on 87K functions, the representation quality is the primary bottleneck, not the decoding strategy.

Capacity and Cross-Package Contamination. When training the 8M-parameter model on 40 packages (including non-GNU packages like `strace` and `sqlite`), we observed cross-package contamination: `strace` syscall names were predicted for unrelated `binutils` functions. Scaling to 25M parameters resolved this; every cross-project package improved. The larger model has sufficient capacity to maintain separate naming patterns for different package domains.

The O0/O2 Generalization Gap. O0-compiled binaries are significantly harder than O2: 41.5% EM vs. 69.8% EM on the test set. O0 code lacks inlining and optimization, producing many small wrapper functions with similar instruction signatures. While our ENDBR64 wrapper resolution rescues the most degenerate cases, the fundamental gap between optimization levels remains an open challenge.

Implications for Binary Analysis Tools. FUNCR’s architecture suggests several integration paths with existing reverse engineering tools. The k -NN inference mode is well-suited for integration as an IDA Pro or Ghidra plugin. The k -NN datastore is an explicit,

updatable knowledge base: when an analyst manually identifies a function, its embedding can be added to the datastore immediately, improving predictions for similar functions *without retraining the model*. By thresholding on cosine similarity, an analyst can control the precision-recall tradeoff.

Comparison with LLM Approaches. The trend in binary analysis is toward ever-larger language models. These approaches achieve strong in-distribution performance but face practical challenges: cost (fine-tuning 7B parameters requires multiple GPUs), inference speed (FUNCR’s k -NN mode is an order of magnitude faster), and updatability (adding new patterns to an LLM requires fine-tuning; adding them to a k -NN datastore is instant). For the specific task of cross-project function name recovery, a well-designed encoder with retrieval-based inference provides a strong, efficient alternative.

8 LIMITATIONS

- (1) **Pure recognizer:** The model cannot compose novel function names. All correct predictions are names seen in training. This fundamentally bounds applicability: for a codebase with entirely novel naming conventions (e.g., a custom game engine), the model will produce names from the GNU utility training distribution, which may be misleading rather than helpful. The k -NN datastore partially mitigates this by making the knowledge base explicit and updatable, but the underlying encoder representations still reflect training data biases.
- (2) **O0/O2 gap:** O0 binaries remain significantly harder (41.5% vs. 69.8% EM). O0 code produces many small, structurally similar functions that the model cannot distinguish.
- (3) **Version sensitivity:** Different versions of the same program produce near-zero accuracy (htop: 0.6% EM despite being in training), because function bodies change across versions while names may stay the same or change unpredictably.
- (4) **BAP dependency:** Our pipeline requires BAP 2.5.0 for IR lifting, which is slower than disassembly-based approaches (e.g., IDA Pro or Ghidra). Processing a single binary takes 30–120 seconds depending on size. A production deployment would benefit from a faster lifter or direct disassembly-based tokenization.
- (5) **Approximate comparison:** Different datasets and evaluation protocols prevent exact comparison with prior work. We mitigate this by following SYMGEN’s deduplication methodology and reporting cross-project metrics, but differences in package selection, compiler versions, and optimization levels introduce confounds.
- (6) **Platform and architecture scope:** We evaluate only x86-64 ELF binaries compiled with GCC on Linux. Windows PE binaries, ARM/MIPS architectures, and other compilers (Clang, MSVC) are not tested. The instruction-type tokenization is architecture-specific and would require reimplementation for other ISAs, though the high-level approach (semantic type abstraction) is transferable.
- (7) **No obfuscation handling:** We assume binaries are not deliberately obfuscated. Control flow flattening, opaque

Table 10: Qualitative prediction examples. “Ext calls” shows external library calls made by the function. Correct predictions are marked with ✓.

True Name	Decoder	k -NN	Ext Calls	Correct?	Notes
xmalloc	xmalloc ✓	xmalloc ✓	malloc, xalloc_die	Both	Shared glibc function, common pattern
hash_pjw	hash_insert	hash_pjw ✓	(none)	k -NN	Decoder confuses similar hash functions
set_program_name	set_program_name ✓	set_prog_name	fprintf, strcmp	Dec	Decoder beats k -NN (rare case)
quotearg_buffer	quotearg_buffer_rest	quotearg_buffer ✓	memcpy, abort	k -NN	Decoder adds hallucinated suffix
diffutils_cleanup	hash_free	xmalloc	free	Neither	Unseen target, both fail
close_stdout	close_stdout ✓	close_stdout ✓	fpending, fclose	Both	Strong ext-call signal

Table 11: Runtime analysis on NVIDIA A100 (40GB).

Phase	Configuration	Time
Pre-training	12M params, 10 epochs	45 min
Training	25M params, 50 epochs	3h 45m
Inference (decoder)	beam $k=5$, batch 32	1.6s/batch
Inference (k -NN index)	87K embeddings, build	2.3s
Inference (k -NN lookup)	cosine sim, per function	<1ms
End-to-end (decoder)	8,973 test functions	7.5 min
End-to-end (k -NN)	8,973 test functions	48s

predicates, and instruction substitution would defeat both the CFG-based graph encoder and the instruction-type tokenization. Evaluating robustness to obfuscation is an important direction for future work.

9 FUTURE WORK

Several directions could extend FUNCR’s capabilities. First, cross-architecture support (ARM, MIPS, RISC-V) would require adapting the instruction-type tokenization to each ISA. Second, breaking through the recognizer ceiling requires compositional generation: predicting names for functions never seen in training. Hybrid approaches combining retrieval (for known functions) with generation (for unknown functions) are promising. Third, the k -NN datastore supports incremental learning: as analysts confirm function names, new entries expand the index without retraining. An active learning framework could suggest which functions to examine next. Fourth, evaluating robustness to binary obfuscation (control flow flattening, opaque predicates) is important for malware analysis applications.

10 CONCLUSION

We presented FUNCR, a graph neural network system for binary function name recovery that achieves 0.804 F1 on a held-out test set (8,973 functions) and 0.519 F1 across 5 completely unseen packages (2,256 functions). Our key contributions are:

- (1) A **multi-context gated fusion mechanism** with conditional bypass that sequentially incorporates external library calls, callee signatures, and caller signatures. Our ablation study reveals the *ext-call paradox*: external calls alone improve test performance but degrade cross-project generalization by 4.3 percentage points. Inter-procedural

callee/caller context resolves this paradox (+12.8pp cross-project EM), demonstrating that naive feature addition can harm generalization and explaining why cascaded fusion with conditional bypass is necessary.

- (2) The finding that **k -NN retrieval outperforms autoregressive decoding** by +2.0pp test EM and +4.7pp cross-project EM with zero retraining. For our encoder architecture, retrieval over embeddings is more effective than generation: the autoregressive decoder’s compositional ability is not just unused but actively harmful (43.8% hallucinated predictions). The encoder representation, not the decoder, is the key component.
- (3) A **comprehensive ablation study** quantifying the contribution of each component across four model variants, from a basic 4.4M-parameter GAT baseline to the full 25M-parameter pretrained system. We demonstrate that multi-context fusion provides +0.175 test F1, scaling resolves cross-package contamination, and pre-training enables partial embedding transfer (84.4%) across vocabulary changes.

Our results demonstrate that binary function name recovery can be effectively solved with a 25M-parameter encoder and simple nearest-neighbor retrieval, achieving competitive or superior performance to systems based on billion-parameter language models at a fraction of the computational cost.

REFERENCES

- [1] J. He, P. Ivanov, P. Tsankov, V. Raychev, and M. Vechev, “Debin: Predicting debug information in stripped binaries,” in *Proc. ACM CCS*, 2018, pp. 1667–1680.
- [2] Y. David, U. Alon, and E. Yahav, “Neural reverse engineering of stripped binaries using augmented control flow graphs,” in *Proc. ACM OOPSLA*, 2020.
- [3] X. Jin, K. Pei, J. Y. Won, and Z. Lin, “SymLM: Predicting function names in stripped binaries via context-sensitive execution-aware code embeddings,” in *Proc. ACM CCS*, 2022, pp. 1631–1645.
- [4] A. Al-Kaswan, T. Ahmed, and P. Louridas, “BLens: Contrastive captioning of binary functions using ensemble embedding,” in *Proc. USENIX Security*, 2025.
- [5] Y. Xu, K. Xu, and D. Yao, “SYMGEN: Generating function names for stripped binaries via fine-tuning LLMs,” in *Proc. NDSS*, 2025.
- [6] L. Jiang, X. Jin, and Z. Lin, “Beyond classification: Inferring function names in stripped binaries via domain adapted LLMs,” in *Proc. NDSS*, 2025.
- [7] X. Xu, Z. Zhang, Z. Su, Z. Huang, S. Feng, Y. Ye, N. Jiang, D. Xie, S. Cheng, L. Tan, and X. Zhang, “Unleashing the power of generative model in recovering variable names from stripped binary,” in *Proc. NDSS*, 2025.
- [8] X. Xu, C. Liu, Q. Feng, H. Yin, L. Song, and D. Song, “Neural network-based graph embedding for cross-platform binary code similarity detection,” in *Proc. ACM CCS*, 2017, pp. 363–376.
- [9] S. H. Ding, B. C. Fung, and P. Charland, “Asm2Vec: Boosting static representation robustness for binary clone search against code obfuscation and compiler optimization,” in *Proc. IEEE S&P*, 2019, pp. 472–489.
- [10] H. Wang, W. Qu, G. Katz, W. Zhu, Z. Gao, H. Qiu, J. Zhuge, and C. Zhang, “jTrans: Jump-aware transformer for binary code similarity detection,” in *Proc. ACM ISSTA*, 2022, pp. 1–13.

- [11] K. Pei, Z. Xuan, J. Yang, S. Jana, and B. Ray, "Trex: Learning execution semantics from micro-traces for binary similarity," in *IEEE Trans. Software Engineering*, 2022.
- [12] Q. Zhu, Z. Feng, L. Zhang, and Y. Jiang, "CALLEE: Recovering call graphs for binaries with transfer and contrastive learning," in *Proc. IEEE S&P*, 2023.
- [13] C. Zhu, Z. Li, A. Xue, A. P. Bajaj, W. Gibbs, Y. Liu, R. Alur, T. Bao, H. Dai, A. Doupé, M. Naik, Y. Shoshitaishvili, R. Wang, and A. Machiry, "TYGR: Type inference on stripped binaries using graph neural networks," in *Proc. USENIX Security*, 2024.
- [14] H. He, X. Lin, Z. Weng, R. Zhao, S. Gan, L. Chen, Y. Ji, J. Wang, and Z. Xue, "Code is not natural language: Unlock the power of semantics-oriented graph representation for binary code similarity detection," in *Proc. USENIX Security*, 2024.
- [15] D. Brumley, I. Jager, T. Avgerinos, and E. J. Schwartz, "BAP: A binary analysis platform," in *Proc. CAV*, 2011, pp. 463–469.
- [16] U. Khandelwal, O. Levy, D. Jurafsky, L. Zettlemoyer, and M. Lewis, "Generalization through memorization: Nearest neighbor language models," in *Proc. ICLR*, 2020.
- [17] U. Khandelwal, A. Fan, D. Jurafsky, L. Zettlemoyer, and M. Lewis, "Nearest neighbor machine translation," in *Proc. ICLR*, 2021.
- [18] S. Borgeaud *et al.*, "Improving language models by retrieving from trillions of tokens," in *Proc. ICML*, 2022.
- [19] S. Bengio, O. Vinyals, N. Jaitly, and N. Shazeer, "Scheduled sampling for sequence prediction with recurrent neural networks," in *Proc. NeurIPS*, 2015.
- [20] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *Proc. ICLR*, 2018.